the tasks that you felt took too long.

A note on weight

One way to do weighting is to look at the number of steps necessary in the completion criteria and use that number. In the example above, note that there are five steps listed in the completion criteria section, and the assigned weight is five. This assigns weight as a proxy for task complexity (which may or may not be appropriate for your study). How you use the weight in your calculations is usually as a multiplier for reporting summative metrics.

As a simple example:

- Take 2 tasks: A and B.
- The score for task A is a perfect 20/20, and the score for task B is a dismal 10/20.
- Now, suppose task A has a weight of 1, and task B has a weight of 2. In this case, B should be weighted twice as much as A.
- When scoring the grade for the combined tasks, multiply each score by its weight. The score for A stays at 20/20, and the score from B becomes 20/40 (10 2 = 20, 20 2 = 40).
- Next, add the totals from the tasks to get the final weighted score (20 + 20 / 20 + 40).
- The total score is 40/60 or .67, not the unweighted average of 30/40 or .75, because we weighed task B more and task B performed worse.

Metrics

The following metrics and definitions are the core of how GitLab performs benchmarking.

Measure	Definition	Scope	Calculation	Example
Completion Rate	% of users who successfully complete the task	Measured per participant, reported per task	Completed = 1, incomplete = 0. Then sum and divide by total participants. Note: completion criteria for each task should be carefully enumerated with your stakeholders prior to running your study	4 completions out of 5 participants for task 3 results in 80% completion rate
Time on task	Time spent from start to finish of task	Measured per participant, reported per task for each participant who meets the task completion criteria	Start timer on verbal cue, stop timer when participant signals that they have completed the task or report that they have no path forward	Avg. time to completion for task 9 = 2:11
CES (Customer Effort Score)	Qualitative measure of perceived effort (1-7, 1 = extreme effort, 7 = effortless)	Measured per task, reported as average	At the end of the task, ask the participant, "On a scale of 1-7 where, 1 is extremely difficult and 7 is extremely easy, how easy was it for you to complete this task?"	Avg. CES for task 13 = 5.9
Error Type Count	Number of the different type of errors or mistakes made during task completion	Measured per task, reported as average or mean	Errors need to be defined alongside the 'happy' or optimal path the user should take	2.6 avg. errors for task X
Error Rate	The number of different types of errors observed over the number of steps in the task	Per task	Take the number of observed types of errors and divide by the number of steps or actions in that task.	Task A has 5 steps. There are 10 participants in the study. Our total steps (denominator) is therefore 50. The numerator is the observed errors across all participants for task A. Suppose there are 20 errors recorded for task A. Error rate is thus 20/50, or 40%

Severity	Judged severity of the problem	Per task, overall	See this handbook page for details	Critical
Grade	A cumulative letter grade portraying the usability of the task overall	per task, overall	see 'Per task grade calculation section below'	C
UMUX lite	Canonically, UMUX lite is a 2-question survey that measures perceived usefulness and usability of a system or product. For benchmarking at GitLab, we tend to use it to measure usability against the specific JTBD in our study.	Collected once per JTBD at end of session.	1 question, on a 7-point Likert scale from strongly disagree to strongly agree.	On a scale of 1-7, where 1 is strongly disagree and 7 is strongly agree, how much to do agree with the statement, "This system helps me perform (insert description of JTBD here)"

Notes on scoring and metrics

If participants don't meet the completion criteria for the task:

- Do not enter time on task for that participant on that specific task. We want to measure the time it takes to successfully complete each task.
- Do include the CES score and error count. We want to understand the experience for everyone who completed the task.
- Mark the task as incomplete, even if a participant believes they have finished the task, but they have not met the completion criteria.
- Mark the task as incomplete, even if a participant believes they have NOT completed the task, but they have met the completion criteria.

For per-task metrics:

- Report all metrics with a 95% confidence interval, unless you have a clear reason to do otherwise.
- For completion rate, use the adjusted Wald calculation for the confidence interval.
- For time on task, use a natural log calculation for confidence interval, and report geometric mean rather than the median as a recommended best practice.
- Precisely defining and accurately counting errors is tricky and does not always need to be performed. If you plan to do it, you must be very clear about the definition for errors in your tasks.
- If you collect error rate, report it per task. Count errors against all possible steps for that task, including multiple recorded errors from the same participant on the same part of a task (for example, clicking the same wrong link several times). This may leave you with an error rate greater than 100%.
- For the Customer Effort Score (CES): If your sample is less than 30, use the population standard deviation to calculate your confidence interval. If N is greater than 30, use the standard deviation calculation. Here's a handy calculator that includes both options and confidence intervals.

For per-workflow overall grade calculation:

- If all per-workflow metrics within a workflow are in one category (e.g., "Fair") but the calculated score is in a different grading category (e.g., "Good"), represent the overall grade as the same category as the metrics (e.g., a workflow score of 80 with "Fair"-category sub-metrics is categorized as "Fair" as well).

Severity calculation

In each session, you will record (per task) the severity number that most closely represents that user's experience as defined on this handbook page. This methodology is similar to the widely-known Nielsen/Norman system, but inverse (where low numbers in our system are of greater severity).

For each incomplete task, rate the severity as 1. For a very painful completion, rate the severity as 2. For a mildly painful completion, rate the severity as 3 (and so on). If the user doesn't encounter any usability issues, rate the severity as 5.

Average the severity score for each task, and assign the overall severity according to the following scores:

Range	Severity
0.0 - 1.9	Severity 1: Blocker
2.0 - 2.9	Severity 2: Critical
3.0 - 3.9	Severity 3: Major
4.0 - 5.0	Severity 4: Low

For example, pretend you have 10 participants in a study. For task A, the severity scores on the individual runs are 1, 4, 1, 2, 2, 3, 4, 5, 1, 3, which sums to 26. Divide by 10, and you'll get the average of 2.6 and a severity assignment of 2: Critical for that task.

Per-task overall grade calculation

For each task, calculate a final grade using the criteria below.

1. Start with the number of completions for each task (for example, you might have 15 completions from 20 participants).
1. Add the average CES score for that task to the number of completions. (Your avg. CES might be 5/7, so 5 + 15 = 20).
1. Next, add the average severity score for that task. (Take the 2.6 from above, for a score of 22.6).
1. Divide this number by the total possible points. Here, the total possible is the number of participants (20), plus the highest possible CES score (7), plus the highest (best) severity score (5). in our case 20 + 7 + 5 = 32). For our example, we have 22.6 points out of 32, so 22.6/32 = 0.70.
1. Multiply the result by 100 (to convert from a decimal) - so 0.70 100 = 70.
1. Use a letter-grading rubric (90-100 = A, 80-89 = B, and so on) to get your final grade: 70 = C.

Here's our above example in table form:

Input	Action	Example
Number off completions (out of total of participants)	-	15 (of 20)
Avg. CES for task (of 7)	add to completions	avg CES = 5, running score = 20
Running total	Divide by total possible	20 / 27 = 0.74

| Decimal result | multiply by 100 | 74 |
| Integer result | map to grading scale (letter grades in this case) | 74 = C |

SUS Companion Analysis

As noted above, triangulating the pain points identified during your benchmark study against the SUS verbatim for your stage helps teams identify which pain points to prioritize by understanding what percentage of the SUS verbatim they address.

Take the following steps to triangulate your benchmark study findings with the SUS verbatim for your stage.

1. Compile a set of verbatims from your stage to use in your analysis.
1. Take out verbatims that are positive-only (e.g., "it's pretty slick with ci") and those that are so brief as not to be informative (e.g., "ci")
1. Take a random sample of 25% of your verbatims to develop your categories.
1. Start with a list of the benchmark pain points for each task and UX theme and add categories as needed to capture new themes like Learnabilty and Documentation. Break-up complex verbatims that reflect multiple themes into separate verbatims. That will help you to apply the categories in a mutually exclusive manner, which is needed to represent the percentage of verbatims that align with different themes.
1. You should have a general category of NA (not applicable) for verbatims that can't be categorized. It makes sense that not all verbatim align with benchmark tasks and themes.
1. After you've developed your categories, apply them to the entire sample of verbatim that you're including in your analysis. Doing this will help you to make sure that you've considered all the possible categories for each verbatim.
1. Report the overall percentage of verbatims that align with benchmark tasks and themes, as well as each individual theme, task, workflow, those from the NA category and any additional categories that you observed (e.g,. Learnability and Documentation) in the appendix to your benchmark report entitled 'SUS Companion Analysis'.

Actionable Insights and Labels

UX Researchers and their teams will identify Actionable Insights, which are issues that articulate the next steps that we will take to address our findings.

Actionable Insights should get the following labels:

- Usability benchmark
- Section::Stage scoped labels to identify the section and stage(s) covered in the study
- Actionable Insight::Product change or Actionable Insight::Exploration needed
- NOTE: Actionable Insight::Product change require a Severity Label to be applied to each issue.
- It's also helpful for stakeholders and for you to rate the severity level of each Actionable Insights so that teams know how to prioritize them. You can use the task level Severity ratings described above to attribute severity levels for each Actionable Insight.

Timeline

Preparing and conducting a benchmarking study takes time. Below is a sample timeline for

starting a typical benchmarking program and running the first study.

Step	Description	Time
Clarify your goals and conduct background research.	Be clear on the why, what, who, when, and how you are going to approach the benchmarking.	1 week
Begin issue	Open a research issue and fill it out.	1 day
Conduct kickoff	Include your stakeholders in helping to refine the scope and direction of the study (focus areas, important metrics, personas, and so on)	1 week
Plan: Overview	Begin your study plan, record the context, reasoning, metrics, personas, and outcomes for your study.	1 week
Plan: Protocol	Write your introduction (exactly what you are going to say), opening questions, and the general flow for your study.	1 week
Plan: Tasks	Enumerate the exact tasks you will measure, the happy path, completion criteria, and weight for each task.	2 weeks
Plan: Test environment	Set up your test environment with any projects and sample data you will use for testing	2 weeks
Recruitment	Open a research recruitment issue at least a month prior to when you wish to run your sessions. A typical benchmarking study uses about 20 participants.	Opening ticket: 1 day. Recruitment itself: 1 month
Run Pilot(s)	The week before your sessions, run 1 or 2 pilot sessions to perfect your protocol and tasks.	1-2 days
Run Sessions	Benchmarking sessions typically last from 90 minutes to two hours. Meaning that for 20 participants, conducting two sessions per day, you are looking at a solid two to four weeks of conducting sessions. Note that you will need to invite more participants than necessary to fill 20 sessions, since not everyone who qualifies will accept the research invite. In order to maximize participant attendance and avoid late cancellations, send reminder emails within 24 hours of each session.	2-4 weeks
Analyze the results.	Calculate metrics, extract recommendations, pull verbatim, put things into Dovetail, and so on.	2 weeks
Prepare the report and share it.	Produce research report, slides, recordings, and so on to disseminate your findings.	2 weeks

You should plan for a full quarter from start to finish for your first benchmarking study, but the time commitment will vary week to week. Benchmarking should not be the only research activity you plan during this time, with the exception of weeks when you are conducting sessions.
Note: This timeline may be significantly reduced in subsequent benchmarking studies for the same tasks and personas.

Usability Benchmarking Toolkit

A few resources to help reduce the start-up cost for a new benchmarking effort:

- Spreadsheet Template - A Google Sheets template for recording your session notes and data, complete with pages and formulas for commonly reported metrics.
- Google Slides Template - A Google Slides template for creating your benchmarking report.
- Google Slides from Q1FY23 Create Benchmarking - The slide deck from the first benchmarking study, from which you can see the structure of the slides and general presentation flow.
- Figjam template for Usability Benchmarking Alignment Workshop - A Figjam template for running

a Usability Benchmarking Alignment Workshop with your stakeholders.
- Example Benchmarking Epic
- Neilsen/Norman on Benchmarking - Process
- Neilsen/Norman on Benchmarking - Metrics

FAQ

Q: How is benchmarking different from the Category Maturity Scorecard or UX Scorecard research?

A: To borrow an analogy, you might think of CMS/UX scorecards as looking at usability through a magnifying glass. Usability benchmarking is like looking at usability through a microscope. Benchmarking is more time intensive. It is not meant to be lightweight. You will speak with more participants (~20) for a longer period of time (~120 minutes, and you will run through more tasks (~25) that might cover more than one JTBD.
Benchmarking reports many of the same metrics (and more!), and those metrics are more likely to be representative due to greater N. At GitLab, we report metrics using industry-standard statistics (confidence intervals at 95%, adjusted Wald calculation for completion rate, and so on). In this sense, you can use benchmarking in concert with CMS or UX scorecards to validate and update those findings.
Benchmarking is also meant to be a program, rather than a one-off study. Benchmarking generates very granular recommendations for usability improvements, and once enough of those recommendations have been implemented, you can run the benchmarking again and start to see trends over time.

Q: Does conducting a usablity benchmarking study mean that I do not need to run a UX Scorecard or CM Scorecard study?

A: No, you should still conduct UX Scorecard and CM Scorecard studies in addition to usability benchmarking studies. All three types of studies are valuable for understanding the users' experience with the product.

Q: How should I view this score vs the UX Scorecard score vs the CM Scorecard score?

A: Since all three (UX scorecard, CMS, and benchmarking) provide an overall letter grade, it is ideal to see all of the grades agree when they are focused on the same task or JTBD.

Q: How often should I conduct usability benchmarking?

A: This is variable based on need and how quickly the recommendations from a previous benchmarking study are implemented. But, generally, there is no reason to conduct benchmarking more than once or twice a year for the same tasks.

Q: Does usability benchmarking have to be conducted on the most current release?

A: Yes. Benchmarking is far too heavy-handed to perform for solution validation of upcoming features, and while you could perform benchmarking on a previous release, the results you gather may already be invalid when you collect them. Given the time commitment, this is highly discouraged.

Q: What GitLab environment should the usability benchmarking be tested on?

A: The UX Researcher on the project can set up a cloud instance of GitLab and create sample data in a project by following the instructions on the UX Cloud Sandbox page. Make sure there is enough sample data to complete all tasks in the benchmarking study when you run your pilot study. You can also ask for help on the ux-cloud-sandbox Slack channel.

Q: How complex/realistic does my testing environment need to be?

A: It depends. Basically you need to look at your task list and go through them, ensuring that every area of the project that you go to (or might reasonably expect participants to go) has something there. Ideally, there's more than 1 item there (file, merge request, comment, environment, deployment, etc.) so that the task time accounts for scanning and searching the desired path. Depending on the areas you're focused on, this may be a lot of sample data, or not so much.

Q:Where does this fit in the product lifecycle?

A: Benchmarking lives at a space bridging the 'improve' stage of the build track and the 'collect ideas/understand the problem' parts of the validation track. Benchmarking is a great way to see how the current product is performing, so it fits well with the 'measure, learn, and improve' part of the cycle. The 'learn' and 'improve' parts of are then taken as inputs into the collect ideas/understand the problem parts of the validation track. So, in benchmarking, we're measuring our current product, validating existing problems we know of, and generating ideas on how to address these problems or generally improve the product.
Note: Benchmarking is somewhat outside of our normal workflow. Since it is so time intensive, it's not something that could or should be a part of every feature or issue, and it's not something we should conduct more than once or twice a year for the same tasks.

Q: For efficiency, can I align a benchmarking study with an upcoming Category Maturity Scorecard study?

A: Yes, this is a great use of benchmarking! You can easily modify your benchmarking study to allow for CMS results. For example, you might run a small number of participants through the CMS protocol in combination with the benchmarking tasks. Because these are both based off of JTBD, the tasks ought to align very well.

Q: What score are we trying to achieve?

A: Rather than aiming for a specific score off the bat, what you really want to see with benchmarking is improvement over time (from one benchmarking study to the next). That being said, benchmarking is metrics heavy, and there are a lot of 'scores' to sift through. Your goal is to get a 10,000-foot view at a glance, with overall grades for each task and each JTBD (A-F, severity 1-5), and also to provide detailed metrics for those who are curious.

Q: Is the scoring dependent on the level of maturity of that category/JTBD?

A: As it stands, no. You should consider maturity during the analysis and when setting expectations, but benchmarking just measures the current experience, however mature it is at the time.

Q: How do Product Designers process and manage the recommendations?

A: For benchmarking, the UX Researcher is responsible for much of the 'processing'. The UX Research team has an FY23 objective to better utilize actionable insights. One of the main outputs of benchmarking is a set of recommendations that will then go through the process of turning into actionable insights or informative insights. Part of this process involves categorizing actionable insights into either 'exploration needed' or 'product change' categories, each of which becomes an issue with this label.

The UX Researcher is responsible for sharing the list of recommendations, along with all other findings from the benchmarking with all relevant stakeholders (specifically Product Designers and Product Managers). The exact structure of this will likely vary between different groups and stages, but part of this process needs to involve the communication and handoff of all 'product change' actionable insights. From there, the team should work together to prioritize these issues and ensure they are completed (which leads to the next question).

It's important to make sure that actionable insights are broken down into smaller, more immediately achievable chunks. Check with your team(s) (during the workshop is a good time) to make sure the actionable insights aren't too big by asking how they could be broken down into smaller issues.

Q: How can I prevent recommendations from going stale (not prioritized/implemented)?

A: This is the critical question. The research team has a KR around research prioritization for the actionable insights marked Actionable Insight::Exploration Needed (needing more research) that are generated from benchmarking. These issues go through a prioritization process for additional research. For those recommendations in the Actionable Insight::Product change category, and thus of more interest to Product Designers and Product Managers, there is a process where a severity label is applied. Generally, Product Designers will assign severity for 'product change' insights, and UX Researchers will assign severity labels for 'exploration needed' insights. For more on the process, refer to this handbook page.

Q: Does the GitLab benchmarking approach involve a competitive analysis?

A: No, not at this time. To use the analogy above, Competitive usability evaulations are also like a microscope, in that UX Researchers look at specific tasks and fine-grained aspects of performance. We might use some of our benchmark tasks in a competitive usability evaluation to see how we measure up to our competitors, but that work is currently out of scope. We first want to introduce the benchmarking approach with our own product, before we look at performance across competitors. Additionally, the first iteration for a competitive usability evaluation at GitLab might be to use them as a magnifying glass for looking at what we can learn from our competition before we put those tasks under the microscope.

Q: How can my team request a benchmarking study?

A: Speak with the UX Researcher on your team to get the process started!

Q: What can I change about the benchmark study protocol?

A: A key charactertistic of a benchmark study is consistentcy. However, flexibility can exist in many ways, such as: driving efficiencies with the data, workshopping the results to collaborate on the recommendations, how you shape your issues/epics, your storytelling, etc.

You can even gather additional data around your tasks, too. If you would like to make a proposal for anything else, such as the metrics captured, start a Google document. In the Google document proposal, include the proposed change, the benefit of the change, the impact of the change, potential concerns you may be introducing, and how might that change get executed.

SECTION: product/ux/ux-research/usability-testing.md

Usability at GitLab

The term "usability" can mean a variety of things. At GitLab, we use the following definition of usability when we conduct user research and design our product:

To be usable, an interactive system should be useful, efficient, effective, satisfying, and learnable.

- Usefulness is the degree to which our product enables a user to achieve his or her goals.
- Efficiency is the quickness with which the user's goal can be accomplished accurately and completely in a period of time.
- Effectiveness refers to the extent to which the interactive system behaves in the way that users expect it to and the ease with which users can use it to do what they intend.
- Learnability is a part of effectiveness and has to do with the user's ability to operate the interactive system to some defined level of competence after some predetermined amount and period of training (which may be no time at all). It can also refer to the ability of infrequent users to relearn the system after periods of inactivity.
- Satisfaction refers to the user's perceptions, feelings, and opinions of the product.

Note: This definition is based on information from the Handbook of Usability Testing and the International Usability and UX Qualification Board curriculum.

Usability testing

Usability testing is the process of evaluating a product experience with representative users. The aim is to observe how users complete a set of tasks and to understand any problems they encounter. Since users often perform tasks differently than expected, this qualitative method helps to uncover why users perform tasks the way that they do, including understanding their motivations and needs. At GitLab, usability testing is part of solution validation.

We also conduct regular Usability Benchmarking studies at GitLab. These are also focused on usability, and are used to set performance and ux benchmarks for specific tasks and workflows across GitLab. As such, they are much more rigorous and time-consuming than a normal usability test ought to be.

Different types of usability testing

Generally speaking, we can differentiate between:

Moderated versus unmoderated usability testing

Moderated tests have a moderator present who guides participants through the tasks. This allows them to have a conversation about their experience, and it helps to find answers to "Why?" questions.

Conversely, users complete unmoderated usability tests on their own without the presence of a moderator. This is helpful when you have a very direct question.

| Moderated usability testing
| Unmoderated usability testing
|
|---

---|---

---|
| Complex questions/WHY? Questions, such as:

  "Why do users experience a problem when using a feature?"
  "Why are users not successful in using a feature?"
  "How do users go about using a feature?" | Direct questions, such as:

  "Do users find the entry point to complete their task?"
 "Do users recognize the new UI element?"
 "Which design option do users prefer?" |

Formative versus summative usability testing

Formative usability tests are continuous evaluations to discover usability problems. They tend to have a smaller scope, such as focusing on one specific feature of a product or just parts of it. Oftentimes, prototypes are used for evaluation.

Summative usability tests tend to be larger in scope and are run with the live product. They are useful if you lack details on why users are experiencing a particular problem or want to validate in the first place how easy it is to use.

Steps for conducting a usability test

1. Establish your research question.
1. Identify the tasks you want to focus on for your usability test.
    - There is no magic number when it comes to how many tasks to include. A guideline is to have 3 - 4 tasks, as this ensures that participants don't get tired and exhausted. Another aspect to consider is that unmoderated test sessions should be 15 - 20 minutes max and moderated sessions 60 minutes max.
    - The key thing to remember when writing your tasks is that they reflect realistic user goals. Taking the JTBD into account when creating your tasks helps keep the focus on user goals. See these tips for writing good tasks that include examples.
1. Define what success looks like.
    - Before testing with users, reach agreement among your stakeholders about what success looks like, such as the target completion rate you are aiming for. For example, a completion rate greater than 80% translates to a Complete or Lovable CM scorecard level. This article from MeasuringU suggests using 78% as a baseline, whereas a minimum of 92% would satisfy a completion rate in the top quartile. However, it's not unreasonable to aim for even 100%; it's up to each team to determine the success criteria required for each study and each feature.
1. Identify your target audience and initiate recruiting.

1. Prepare your prototype or demo environment.
   - Consider which mode to test in: Use light mode by default for usability testing to help maintain testing efficiency while ensuring consistency across studies. Testing in dark mode should be a deliberate methodological choice and only be considered when the mode is the primary focus of your research, when mode variations might significantly influence your study's results, or when catering to specific user preferences or accessibility requirements. Examples of when dark mode testing should be considered: Investigating visual hierarchy's impact across modes; examining task performance variations in different modes; analyzing mode adaptation to environmental contexts such as lighting; exploring user engagement patterns across modes; examining mode compatibility and technical integration within digital ecosystems.
1. Write your test script, including tasks and metrics to collect.
   - Set up your test to measure these usability factors, as this will help to measure improvements consistently over time and to assess their impact on system usability. There are many other metrics that you can measure to understand usability problems, such as error rates or number of times a user needed help. If you find them helpful for your research topic, feel free to use them.
   - Remind participants to think aloud as they go through the tasks, especially in an unmoderated test.
   - Take a look at these more detailed tips and tricks on how to write an excellent usability test script.
   - If you run an unmoderated usability test on UserTesting, create a test from the usability metrics template, located in the "Account templates" section. UserTesting has native options for task success and difficulty to capture metrics that are similar to ours, but these are not the same usability metrics needed for our studies. Please use the options listed in this template instead.
   - If you run an unmoderated usability test that will run under 5 minutes, toggle the Short test option on when building your test plan.
1. Run a pilot session to test the usability test.
1. Analyze your research data
   - For each task, synthesize how many users succeeded or failed, and why they failed.
   - For each task, calculate the average score for the efficiency question. Look for patterns on why they gave a score.
   - Calculate the average score for the satisfaction and usefulness questions. Look for patterns on why they gave a score.
   - Note down any other interesting observations you had.
1. Document your insights in Dovetail. If you have actionable insights, ensure they are also documented in the GitLab UX Research project.
1. Decide on the next steps.
   - Any actionable insights require a follow up. Work with your counterparts to determine priority for the identified usability problems. Remember to conduct another usability study to validate your proposed solution.

Usability factors to measure

To learn more about these factors, see our definition of usability.

Effectiveness

- What's being measured?
  - Effectiveness can be determined by calculating the pass rate. This shows the success or

completion rate of each task.
- How is it measured?
  - The pass rate can be determined by dividing the number of participants that were able to complete the task by the total number of participants.
  - Ensure that you are observing and documenting why participants are failing as well.
- Why are we measuring it?
  - We want users to succeed in reaching their goals. Understanding why and where they fail will help us improve the experience.

Efficiency

- What's being measured?
  - Efficiency can be measured by understanding the participants perception of how easy a task is.
- How is it measured?
  - After each task, ask the Single Ease Question.
    - "Overall, this task was·"
 - Extremely difficult
 - Difficult
 - Neither easy nor difficult
 - Easy
 - Extremely easy
 "Why?"
  - Ensure that you are asking why participants rated it this way right after they reply.
- Why are we measuring it?
  - We also collect this during Category Maturity Scorecards (CM Scorecards). Collecting it as part of a usability test gives a prediction and pulse check for a later CM Scorecards study.
  - It is important to understand the participants' reasons for giving a rating, especially in situations where a rating is low. This is especially important in usability testing when the number of participants is also low.

Satisfaction

- What's being measured?
  - Satisfaction can be determined by understanding the participants' perception of how the experience is.
- How is it measured?
  - After all tasks are completed, ask the Satisfaction rating question.
    - "How would you rate the quality of this experience?"
 - Extremely good
 - Good
 - Neither good nor bad
 - Bad
 - Extremely bad
  - Ensure that you are asking why participants rated it this way right after they reply.
- Why are we measuring it?
  - We also collect this during CM Scorecards. Collecting it as part of a usability test gives a prediction and pulse check for a later CM Scorecards study.
  - It is important to understand the participants' reasons for giving a rating. This is especially important in usability testing when the number of participants is also low.

Usefulness

- What's being measured?
  - Usefulness can be determined by understanding the participants perception of how much the tested features would enable them to achieve their goals.
- How is it measured?
  - After all tasks are completed, ask our adjusted UMUX Lite question.
    - "You just experienced our implementation of <Feature/Scenario>. How would you agree or

disagree with the following statement: <Feature/Scenario> has the capabilities I need for what I need to do in my own work."
 - Strongly agree
 - Agree
 - Neither agree nor disagree
 - Disagree
 - Strongly disagree
  - Ensure that you are asking why participants rated it this way right after they reply.
- Why are we measuring it?
  - We also collect this during CM Scorecards. Collecting it as part of a usability test gives a prediction and pulse check for a later CM Scorecards study.
  - This score highly correlates to the System Usability Scale (SUS) score, one of our regular performance indicators.
  - It is important to understand the participants' reasons for giving a rating, especially in situations where a rating is low. This is especially important in usability testing when the number of participants is also low.

How usability testing relates to Category Maturity Scorecards (CM Scorecards)

Usability testing happens before you conduct a CM Scorecard. Usability testing helps to identify the majority of problems users encounter, the reasons for these issues, and their impact when using GitLab, so we know what to improve.

If you run a CM Scorecard without prior usability testing, you will likely identify some of the usability problems users experience. However, the effort and rigor connected with the Category Maturity Scorecard to measure objectively the current maturity of the product doesn't justify skipping usability testing.
In addition, during usability testing you have opportunities to engage with participants directly and dive deeper into understanding their behaviors and problems. The Category Maturity Scorecard process does not allow for such interactions as it's designed to capture unbiased metrics and self-reported user sentiment.

Frequently Asked Questions

Why should I not ask in user testing if a participant completed a task successfully?

It's a self-reported measure which may or may not be true. If a participant indicated they completed a task successfully, you still need to check if they really completed it based on how you defined success. It's important to capture only necessary data from users and considering you need to double-check their response with actual behaviour, we suggest to leave it out in the first place.

Why should I not use UserTesting.com's native task metric difficulty to assess Efficiency? Isn't it the same?

UserTesting.com's task metric difficulty is similar but it uses a different rating scale than the Single Ease Question. Currently, there is no option to change the scale labels in UserTesting.com to align with ours.

SECTION: product/ux/ux-research/user-story-mapping.md

What is user story mapping?

User Story mapping is a powerful way to visualize how people are using your product or feature holistically and organize individual stories to that journey. A story map can even be annotated to indicate which user stories need to be included in each release, making it a valuable tool for PMs and Designers to use during planning.

Much like a User Journey Map,
a Story Map outlines a workflow and breaks it in to individual steps. Then, each story (which is likely issues, in GitLab's case), are organized around the individual steps in the workflow.

From there, the stories can be organized in priority ranking, or grouped in to milestones to make it easier to think about planning.

Resources

Here are some resources to learn more about story mapping:

1. Agilevelocity Story Mapping 101
1. Easyagile Building Story Maps PDF
1. Jeff Patton Story Map Concepts PDF
1. GitLab story map training video, slide deck
1. GitLab example story mapping video

SECTION: product/ux/ux-research/using-rite-to-test-navigation.md

Rapid Iterative Testing and Evaluation (RITE) is a usability testing method in which you evaluate a prototype multiple times in a rapid and iterative manner. The goal is not to determine statistical validity, but to observe behaviors, learn insights, and iterate rapidly. This is different from traditional usability testing, because you iterate during testing rather than waiting until the end to make changes. This method is intended to identify and fix as many usability issues as possible while verifying the effectiveness of changes made within testing. This will result in a navigation prototype that we can be highly confident about (in the context of usability), which helps to remove uncertainty around whether a proposed solution will be usable.

Why should I use this method?

You have a navigation design concept that you want to move forward with, and you want to assess the basic usability of it. You want to know what's working well and not working well with your prototype early in the solution validation process and test changes quickly. Using the RITE method will enable you to evaluate navigation designs quickly to encourage iteration.

Goal: Learn if users can navigate to a task with the new navigation design, assess basic usability, identify aspects works/don't work with the design, and obtain additional insight such as initial impressions and other qualitative feedback.

The types of research questions this method can answer:

- Do users know where to go to complete a task with your navigation design? Why or why not?
- Can users complete tasks with your navigation design? Why or why not?
- Can users discover something new in the UI with your navigation design? Why or why not?
- Does the proposed layout in your navigation design make sense? Why or why not?
- Do users know where they are in the application with your navigation design? Why or why not?
- Are UI elements standing out to users in your navigation design? Why or why not?

Methodology details

Type of facilitation: This testing may be moderated or unmoderated, depending on how much you need to probe on any feedback provided by participants, their observed behavior, and their ability to understand the prototype on their own. Participants should utilize the think-aloud method.

Fidelity required: A prototype with enough fidelity to test the basic interactions should be used for this kind of testing. For example, if you want to assess whether participants can navigate to an area, you would need an interactive prototype that allows participants to complete that task, and ideally be able to click a couple of "wrong" areas.

Recommended sample size: It varies, depending on whether or not you continue to find issues. With the RITE method, you start with 3 participants, fix any usability issues that were found with the initial design, and test with 2 additional participants for a total of 5 participants.
If no additional issues are found you're done. If additional issues are found, add another 3 participants until no further issues are found.

What to capture:

- How participants navigate to complete tasks
  - Common paths (whether correct or not)
    - Why they chose that path via qualitative feedback
- What impacted their success
  - What made the task difficult/easy?
  - If there was difficulty, was the participant able to recover? If so, how much effort was needed? why?
  - Did participants notice something (we wanted them to notice?)
- Usability Issues
  - Did participants struggle to find something in the navigation? why?
  - Did a navigation item label confuse participants? why?
  - Did the navigation item not behave as expected (such as searching in the top menu)? Why?
  - Issues should be classified in each session following this format:

Usability problems identified in a session are categorized into the following categories:

Category	Description	What to do
A	Issues with an obvious cause and an obvious solution that can be implemented in the prototype quickly, such as text changes, re-labeling buttons.	Implement solutions immediately and this will be the version for the next test session.
B	Issues with an obvious cause and an obvious solution that can't be implemented quickly/by the time of the next test session.	Start working on the solution to address these issues and

test in an upcoming session.|
|C|Issues with no obvious cause or due to other factors, such as task instructions. |Collect more data in the upcoming session until they can be promoted to Category A or B.|

Qualitative feedback about their experience

- Post-task questions:
  - Do you have any feedback on the activity you just worked on?
  - Was anything particularly easy or difficult (if not already mentioned)? why?
  - (If moderated), probe on specific things the user was doing during the task.
- Post-test questions:
  - What did you like or dislike about the experience?
  - (If moderated), probe on any interesting/unclear behaviors/observations observed during the test.

How to run a RITE study

Refer to the RITE handbook page for details.

Recommended platform: UserTesting for unmoderated studies, Zoom for moderated studies.

Reporting results

To report brief/initial findings in Slack or in an Issue, please use the following format:

After the first 3 participants:

- Number of usability issues found
- Usability issues found with a brief description of each
  - For example, "2 out of 3 participants struggled to find a recent project with the new placement of the [button or element name]."
- Themes around behaviors that led to the usability issue
  - For example, "2 participants expected to find it [somewhere else]
- Note any other interesting observations or feedback
  - For example, "All 3 participants expressed confusion around the change with [X] because·"
- Note any issues that you plan to address before running the next 2 participants.

After the next 2 participants:

- If additional issues are found:
  - Note the number of usability issues found, provide descriptions of each issue, and any themes around behaviors (same as format above for the first 3 participants).
  - Note any other interesting observations or feedback.
  - Note any issues that you plan to address before running 3 additional participants (repeat adding participants as indicated above).
- If no additional issues are found:
  - Note that no additional usability issues were found
  - Themes around behaviors and any associated feedback
    - For example, "All 3 participants were able to find the [name or element name] easily." The placement is where they expected to find it, and they felt the label was intuitive.

In addition, if there is any context that you can provide in your interpretation of the results that is helpful. For example, if participants selected Merge Requests in the left sidebar, you can add that the common incorrect first click in this area may be due to the task calling out a merge request.

SECTION: product/ux/ux-research/ux-research-growth-and-development.md

Based on past Pulse Survey results for UX Research Team Members, we've added two components to (the existing GitLab Career Development process) that focus on: (1) discussing development in an individual's current level and (2) providing targeted guidance to incorporate into the Individual Development Plan (IDP). The goal is to help identify an individual's greatest opportunities for growth, and then enable them to work with their manager to identify projects that align with those development areas.

Level Progression Check-In

A Level Progression Check-In is an informal exercise that takes a UX Research Team Member's current and next level, compares them side by side, and gives the manager structure to provide feedback on each of the responsibilities in their current level as outlined in the job family. The goal of this exercise is to provide a structure to give feedback inline with the responsibilities outlined in the job family. It should give the UX Research Team Member a sense of how they're performing, where they are doing well, and where they have opportunities to improve.

Initially, the manager will go through each responsibility and provide a written summary of how the individual is performing, including suggestions for improvements. The individual can then decide how to act upon that feedback; for example, designing a goal around it. Then, each month during a 1:1, the manager and the UX Research Team Member will discuss how they have progressed since the last conversation and update the Level Progression Check-In document in the notes column.

Promotion Document

A promotion document is created for each UX Research Team Member. Together with the manager, the individual will maintain a working version of this document. UX Research Team Members will work closely with their manager to identify impactful examples that could be used as justification for a promotion as they go.

Each month the UX Research Team Member should update their document and have a conversation with their manager in their 1:1.

Career Development Conversations

Each month, the manager will hold a career development conversation. This conversation should be held during their 1:1 and touch on:

- Individual development plan
- Level progression check-in

- Promotion document

SECTION: product/ux/ux-research/ux-researcher-pairings.md

Inspired by the Product Designer pairing program, UX Researchers can take part in their own pairing offering: UX Research pairings.

UX Research pairings

UX Research pairings gives UX Researchers a consistent partner to share ideas with on research approaches, addressing challenges, reviewing test plans and reports, and gaining experience in delivering feedback. It also gives UX Researchers exposure to product areas outside of their own. Some details:

- UX Research pairings are completely optional and are not required.
- If a UX Researcher is interested in this offering, they must take the initiative and seek out another UX Researcher to pair up with. They must also be willing to take part in the offering, too.
- By design, the UX Research pairing is unstructured. This leaves the cadence, feedback topics, etc up to the pairing to shape as they wish. It also means that pairs decide what to share with each other and when to share it.
- A pairing can last up to six months. After that time, it's encouraged to identify new pairings should the UX Researchers wish to continue in the offering.

Examples of UX Research pairings

UX Research pairings can look similiar or different from one another. Some possible examples:

- Pairings met twice a week, synchronously, to work together on a usability benchmark test plan where they focused on working out logical tasks for participants to complete. Once that was delivered to and approved by stakeholders, the pairing cadence was reduced to ad-hoc through asynchronous meetings.
- Pairings met weekly, asynchronously, mainly to obtain feedback on test plans, data analysis, final reports, etc. The overall goal of this pairing is to work and deliver results in a more standard fashion.
- The pairings decided to work on an OKR together. To effectively successfully land that, they met weekly, both asynchronously and synchronously, during the quarter. Together, they planned the measures and deliverable of the OKR, while providing each other with feedback and guidance along the way.

The current UX Research pairing rotation list

If you're part of a pairing, please document it in the table below:

UX Researcher	UX Researcher pairing	Pairing details	End date
Will Leidheiser	Danika Teverovsky	Learn about PNPS process from start to finish through a mixture of sync calls and async recordings	2023-04-30

| Michael Oliver | Danika Teverovsky | Develop more skills for discovering JTBD and how to use the framework to find strategic opportunities at GitLab | 2023-10-31 |
| | | | | |

SECTION: product/ux/ux-research/writing-usability-testing-script.md

What is a usability study?

A usability study, or usability testing, employs a test script to ensure a systematic approach for:

- Testing whether users can carry out scripted tasks successfully
- Finding what their preferences for completing a task are
- Uncovering opportunities to improve the product

What is a testing script?

At the heart of a usability study stands the testing script, or test script, which the moderator follows during test sessions in order to execute task based usability testing and ensure consistency.

A test script:

- Helps to make sure you will be covering what you set out to cover. It helps to ensure that you meet your usability study research objectives and that you come out of the study with the answers you need to make progress on your project.
- Keeps you consistent between sessions. It's important to make sure you ask the same questions of all of your participants to maintain a rigorous methodology. It also makes your life easier during analysis.
- Assists with collaboration. In particular, it helps others more easily review your work.
- Helps you keep time during sessions. It's important to keep your research sessions to the promised length.

It is virtually impossible to practice solid usability study research without having a test script. Don't go in without one.

How to write your first task based usability testing draft

Use this Usability Testing Script template (internal access only) as your starting point for writing a first draft for your usability study:

A usability study script typically follows 4 main parts:

1. Introduction
1. Warm-up questions
1. Tasks
1. Wrap-up questions and closing words

Here is a bit about each part (you may want to read this while going over the template).

Introduction

This is where you introduce yourself and other attending members of the team. You also tell the participant of the usability study how the session is going to go and double check that you have their consent for recording and sharing the session.

Use the introduction to build rapport with the participant of the usability study. People are often hesitant, nervous, or even a little standoffish at the beginning of a research session. Simple comments such as "Oh, I've been to where you live and I loved it" can go a long way to making people feel comfortable and helping the study to run more smoothly.

Remember to distance yourself from the solution you are testing. Do not present yourself as the person who is involved in creating the concept that's being tested (even if you were). Emphasize that the participant won't hurt your feelings regardless of what they say during the testing. Remind them that we are testing the experience and not them. Also remind them that the main purpose of this exercise is to receive their candid feedback.

Warm-up questions

Warm-up questions are meant to further break the ice, as well as get relevant background information on the participant.

Here are some standard warm up questions to consider:

- What's your current role?
- How long have you been in that role?
- Do you have experience dealing with [topic of study]?
- Have you ever used GitLab? If so, what for?

Tip 1: Even if you don't have anything you want to ask, have at least 2 quick questions here, as it will help to break the ice and make the participant feel more at ease.

Tip 2: Consider asking questions that will help you to understand the participant's mental model and expectations prior to interacting with your designs (for example, "We have a page called Security Dashboard. What tasks would you expect to be able to accomplish with the help of that page?").

Tip 3: If needed and you have the time for it, you might include some additional interview questions here that would benefit either this study or a different one. (If it's a different study, make sure it shares the same participant profile with this one!) In particular, consider asking questions that might be used later on as part of a persona or a JTBD study (for example, "What would you say are your top 3 tasks?").

Tasks

This is the heart of the usability study script and the part that takes the longest to write. Usability studies usually (but not always) consist of 3·4 tasks.

How to define good tasks

When sitting down to write your tasks, you should already have defined your research goals, objectives and hypotheses. To form good usability tests, start by going over your research objectives that detail what you and your stakeholders want to get out of the study, and consider how to best translate them into user tasks.

You should also have clear, agreed-upon measures to use when determining how a task, experience, or design/prototype performed. You need to define what Pass/Fail looks like, so you can determine when a participant has successfully completed the usability task.

The key thing to remember when moving from objectives to tasks is that your tasks should reflect realistic user goals.

For example, perhaps one of your objectives entails finding out whether users can quickly locate a CTA (call to action). But since no user ever goes into a website with the goal of locating a button, your task should never be, "Can you find and click the button?" Rather, it should be about why the participant might want to click the button in the first place (for example, "You want to enable feature X for your project. Can you do that please?").

Another objective might be identifying potential improvements to the flow. Since real users don't generally go into websites with the sole purpose of finding faults in it, instead of asking the participant, "Can you please complete the following steps and tell me what we should improve?", ask them to perform a task that would necessitate them attempting the flow, and observe them carefully to see where they fail, struggle, or hesitate.

Tip: Avoid using any words that are in the UI. When a task is written with easily identifiable words in the UI, participants tend to look for those words in the UI to complete the task.

Finally, pay close attention to how you phrase your tasks to avoid bias, leading the participant, and other common pitfalls.

- It is important to balance clarity and leading words when writing a scenario. The words you use should be words that are familiar to your participants. If you are not sure what those words should be, you should reach out a Researcher for guidance and feedback.
- If you use words which your participants use and still find they are scanning the interface to complete the task(s), change how you present the task. Start each task by asking the participant to describe the thing you are wanting them to do in the system and walk you through how they would normally do that task. Then you can use the words the participant has just used with you to direct them to do the task.
- It is imperative to ensure your participant fully understands the task given to them before they begin the task. As the facilitator, you have to balance the questions your participant asks when seeking clarification on the task, with not providing information which could be used to successfully complete the task.
- A usability study, or usability tests, should be focused on evaluating if the participant can accomplish the task given, not exploring how they might understand the task. If your participant can not proceed during some point of the task, ask them how they would normally proceed and encourage them to try that option.

To learn more about writing good tasks, we highly recommend reading Write Better Qualitative Usability Tasks: Top 10 Mistakes to Avoid.

Tip 1: For each task, add a link in your script for the prototype/webpage that's relevant for that task. Not only will it help your teammates who will review the script to understand what the task is about, but it will also allow you to quickly resend the relevant link should the participant need it again.

Tip 2: Consider noting under each task, in light gray, 'What we expect them to do' (for example, "Follow the CI pipeline and go into the SAST job output") to remind the moderator of the possible paths for completing the task. This will assist the moderator in helping participants to recover, in case they fail a task which is a prerequisite for the following task.

How to order your tasks

- If your tasks can build on top of each other in a sensible way, make sure you order your tasks to reflect that. For example, Task 1 could be around navigating to a certain page and Task 2 around reviewing that page.
- Cluster together tasks that belong to the same topic or area of the product.
- As a general rule, start with the tasks that matter most to you. It will take time mastering moderating usability sessions, and it is common to fall behind on time when you're just starting out. Therefore, start with what matters most to you, and leave what's merely nice to have to the end of the test.

How to structure each usability testing scenario

For each task, consider whether some setup is required to provide context and appropriate motivation for the participant. If so, describe a relevant scenario prior to giving the task.
Let's look at some usability test cases examples:

Scenario: "Let's say this is a project you're working on, and you just committed some new code."
Task: "Please test to see whether that code contains any security vulnerabilities."

Then, consider adding some more specific questions and prompts that the moderator can utilize as the task unfolds, in case the participant doesn't bring up these topics on his or her own. Examples:

- What are you seeing on this page?
- What is this page about?
- Is SAST running right now?
- What would you do next, if anything?

Wrap-up questions and closing words

Here, you can get the participant's broad impressions about what they saw and experienced. These are some standard questions to consider:

- What do you think about this process you just went through?
- How does what you just experienced with GitLab fare in comparison to the tool you normally use?
- Anything else you'd like to add?

You can also include qualitative survey questions. Make sure to follow up with "Why did you choose that answer?" For example:

1. Completing the [insert task] was easy.
    - I completely agree
    - I somewhat agree
    - I'm neutral
    - I somewhat disagree
    - I completely disagree

Conclude the script with thanking your participant and mentioning when they are expected to get their compensation.

You've completed your draft script. Now what?

1. Review your usability study draft

Once your draft is more or less done, give it another read and ask yourself:

- Does it cover the project's objectives?
- Does it make sense timewise? Estimating time will get easier with experience. Keep in mind that usability tests should normally take between 30·45 minutes.

2. Let others review your draft

Edit as needed based on feedback received from your stakeholders/teammates.

3. Test the test

Run a pilot test with a colleague or internal participant to make sure your task instructions are clear and that you're keeping time. Edit as needed, and notify your stakeholders of any big changes.

A crash course on remote, moderated usability testing

{{< youtube "5MvpxvN9vLU" >}}

SECTION: product/ux/ux-research/community-contributions-for-actionable-insights/index.md

UX Research is most valuable when insights are acted upon and result in implementation. As part of UX Research deliverables, Actionable Insight issues are created which allow product teams to pick up and fix the issues that UX research uncovers. What's great about GitLab is that everyone can contribute, not just our internal product teams. Engaging the wider GitLab community and asking for their support may help to get Actionable Insights addressed more quickly and with that improve the experience of using GitLab. One way to help surface those insights is by writing a blog post.

The process below describes how UX Researchers can create blog posts calling for community

contributions to implement solutions that address user problems identified in UX Research.

Who is involved and what are their responsibilities?

While UX Researchers are the DRI for creating the blog post, collaboration with the following team members is needed:

- Product Designers
- Product Managers
- Engineering Managers
- Editorial Team
- Contributor Success Team

Responsibilities

The table below outlines key responsibilities. Refer to the blog creation workflow for more details.

Role	Responsibility
Product Designer	- Designs or reviews, and approves the solution to address the user problem outlined in the Actionable Insight issue (in collaboration with Product Managers and Engineering Managers) - Conducts Solution Validation as needed - Applies "Ready for Blog" label
UX Researcher	- Writes blog post including links to Actionable Insights - Applies label(s) "Seeking community contribution" and "quick win" (if known) - Opens issue for blog post proposal
Product Manager	- Collaborates with Product Designer on solution to address identified user problem - Reviews and approves blog post
Engineering Manager	- Collaborates with Product Designer on solution to address identified user problem
Code Contributor PM	- Reviews selected Actionable Insights and ensure they are suitable for community contributions - Reviews blog post for proper language and focus for community contributions
Editorial Team	- Reviews, makes final edits and approves blog post - Schedules and publishes blog post

What does the blog creation workflow look like?

View the workflow in Figjam.

What content should I include in my blog post?

It's recommended to include the fo